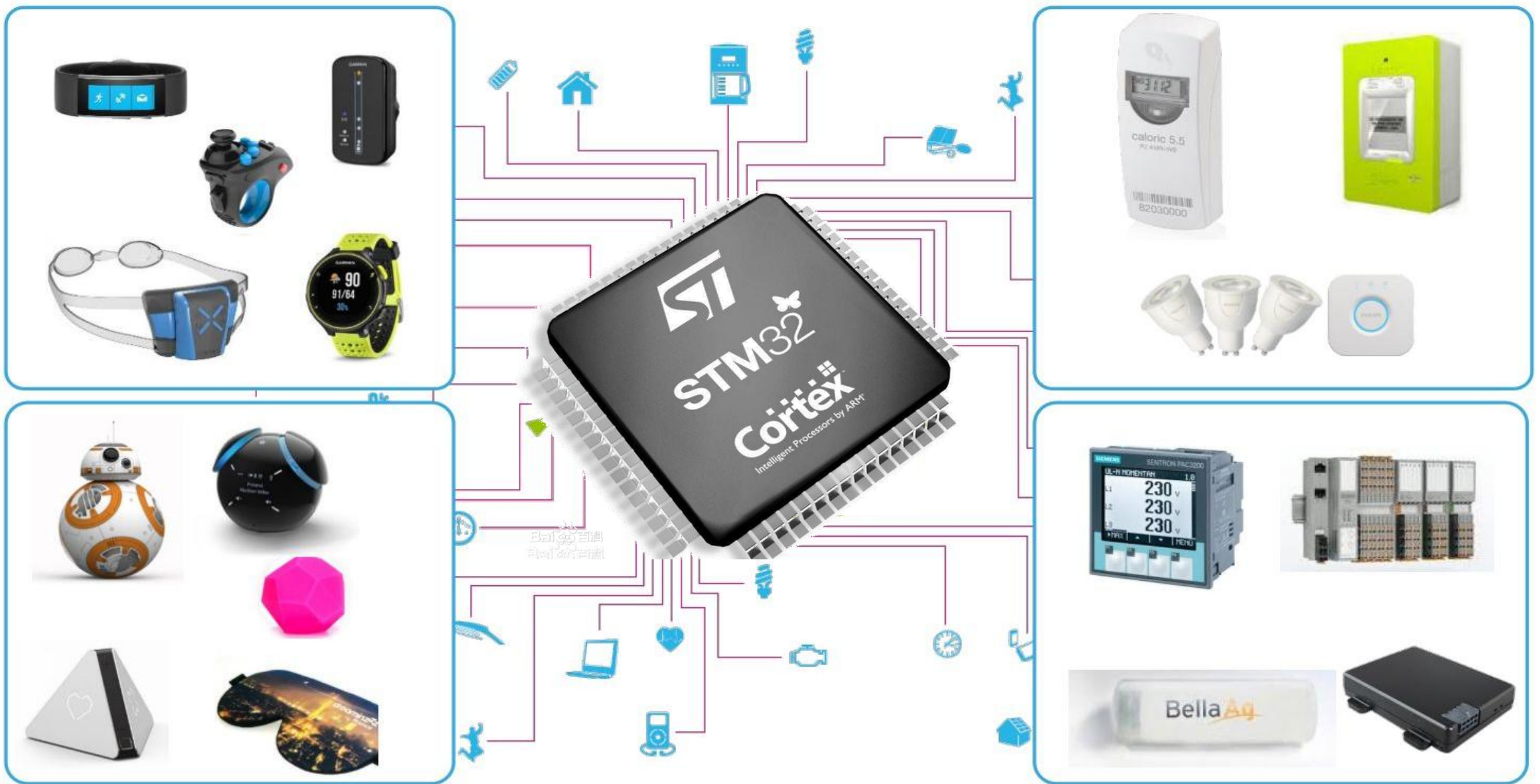




Instructor : WangXiaojing

Email : wangxj@sustech.edu.cn



Lecture3

Firmware Library

Topics

- Depth Analysis of `GPIO_Init()`
- Firmware Library
- `Template_FWLib_Project`

Part I : Depth Analysis of GPIO_Init()

Explain the Function

——`GPIO_Init(GPIOB, &GPIO_InitStructure);`

main.c

```
// 打开 GPIOB 端口的时钟  
RCC->APB2ENR |= (1) << 3);
```

```
// 配置IO口为输出
```

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

```
// 控制 ODR 寄存器
```

```
GPIO_ResetBits(GPIOB, GPIO_Pin_5); //LED on  
//GPIO_SetBits(GPIOB, GPIO_Pin_5); //LED off
```



```
GPIOB->CRL &= 0xFF0FFFFFF;  
GPIOB->CRL |= 0x00100000;
```

```

GPIO_InitTypeDef GPIO_InitStructure;
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
GPIO_Init(GPIOB, &GPIO_InitStructure);

```



```

GPIOB->CRL &= 0xFF0FFFFFF;
GPIOB->CRL |= 0x00100000;

```

```

typedef struct
{
    uint16_t GPIO_Pin;
    GPIO_Speed_TypeDef GPIO_Speed;
    GPIO_Mode_TypeDef GPIO_Mode;
}GPIO_InitTypeDef;

```

```
typedef enum
```

```

{
    GPIO_Speed_10MHz = 1,    // 10MHz    (01)b
    GPIO_Speed_2MHz,        // 2MHz    (10)b
    GPIO_Speed_50MHz        // 50MHz    (11)b
}GPIO_Speed_TypeDef;

```

```
typedef enum
```

```

{ GPIO_Mode_AIN = 0x0,      // 模拟输入    (0000 0000)b
  GPIO_Mode_IN_FLOATING = 0x04, // 浮空输入    (0000 0100)b
  GPIO_Mode_IPD = 0x28,     // 下拉输入    (0010 1000)b
  GPIO_Mode_IPU = 0x48,     // 上拉输入    (0100 1000)b

  GPIO_Mode_Out_OD = 0x14,  // 开漏输出    (0001 0100)b
  GPIO_Mode_Out_PP = 0x10,  // 推挽输出    (0001 0000)b
  GPIO_Mode_AF_OD = 0x1C,   // 复用开漏输出 (0001 1100)b
  GPIO_Mode_AF_PP = 0x18    // 复用推挽输出 (0001 1000)b
}GPIO_Mode_TypeDef;

```

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register		
General purpose output	Push-pull	0	0	01 10 11 see <i>Table 21</i>		0 or 1		
	Open-drain		1			0 or 1		
Alternate Function output	Push-pull	1	0					Don't care
	Open-drain		1					Don't care
Input	Analog	0	0	00				Don't care
	Input floating		1					Don't care
	Input pull-down	1	0			0		
	Input pull-up					1		

Table 21. Output MODE bits

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

In input mode (MODE[1:0]=00):

00: Analog mode
 01: Floating input (reset state)
 10: Input with pull-up / pull-down
 11: Reserved

In output mode (MODE[1:0] > 00):

00: General purpose output push-pull
 01: General purpose output Open-drain
 10: Alternate function output Push-pull
 11: Alternate function output Open-drain

STM32F103 GPIO 引脚工作模式真值表分析											
			十六进制	二进制							
				bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
				上/下拉		输入/出		具体工作模式 对照寄存器说明			
GPIO_Mode_TypeDef	GPIO_Mode_AIN	模拟输入	0X00	0	0	0	0	0	0	0	0
	GPIO_Mode_IN_FLOATING	浮空输入	0X04	0	0	0	0	0	1	0	0
	GPIO_Mode_IPD	下拉输入	0X28	0	0	1	0	1	0	0	0
	GPIO_Mode_IPU	上拉输入	0X48	0	1	0	0	1	0	0	0
	GPIO_Mode_Out_OD	开漏输出	0X14	0	0	0	1	0	1	0	0
	GPIO_Mode_Out_PP	推挽输出	0X10	0	0	0	1	0	0	0	0
	GPIO_Mode_AF_OD	复用开漏输出	0X1C	0	0	0	1	1	1	0	0
	GPIO_Mode_AF_PP	复用推挽输出	0X18	0	0	0	1	1	0	0	0
这8个宏的高4bit可随意设置，只要能在程序上帮助判断出模式即可，真正写到寄存器的值是bit2和bit3											

GPIO_Mode_TypeDef GPIO_Mode

STM32F103 GPIO 引脚工作模式真值表分析

			十六进制	二进制							
				bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
					上/下拉		输入/出	具体工作模式 对照寄存器说明			
GPIO_Mode_TypeDef	GPIO_Mode_AIN	模拟输入	0X00	0	0	0	0	0	0	0	0
	GPIO_Mode_IN_FLOATING	浮空输入	0X04	0	0	0	0	0	1	0	0
	GPIO_Mode_IPD	下拉输入	0X28	0	0	1	0	1	0	0	0
	GPIO_Mode_IPU	上拉输入	0X48	0	1	0	0	1	0	0	0
	GPIO_Mode_Out_OD	开漏输出	0X14	0	0	0	1	0	1	0	0
	GPIO_Mode_Out_PP	推挽输出	0X10	0	0	0	1	0	0	0	0
	GPIO_Mode_AF_OD	复用开漏输出	0X1C	0	0	0	1	1	1	0	0
	GPIO_Mode_AF_PP	复用推挽输出	0X18	0	0	0	1	1	0	0	0
这8个宏的高4bit可随意设置，只要能在程序上帮助判断出模式即可，真正写到寄存器的值是bit2和bit3											

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

```
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5; //0x0020;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //0x0010;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz; //0x0001
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

两个参数都是结构体指针，所以要传进来的是地址。

```
3 void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)
4 {
5     uint32_t currentmode = 0x00, currentpin = 0x00, pinpos = 0x00, pos = 0x00;
6     uint32_t tmpreg = 0x00, pinmask = 0x00;
7
8     /*----- GPIO 模式配置 -----*/
9     // 把输入参数GPIO_Mode的低四位暂存在currentmode
10    currentmode = ((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x0F);
11
12    // bit4是1表示输出，bit4是0则是输入
13    // 判断bit4是1还是0，即首先判断是输入还是输出模式
14    if (((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x10)) != 0x00
15    {
16        /*----- GPIO CRL 寄存器配置 CRL寄存器控制着低8位IO -----*/
17        // 配置端口低8位，即Pin0~Pin7
18        if (((uint32_t)GPIO_InitStruct->GPIO_Pin & ((uint32_t)0x00FF)) != 0x00)
19        {
20            /*----- GPIO CRH 寄存器配置 CRH寄存器控制着高8位IO -----*/
21            // 配置端口高8位，即Pin8~Pin15
22            if (GPIO_InitStruct->GPIO_Pin > 0x00FF)
23            {
24            }
25        }
26    }
27 }
```

- 1、确定currentmode的值
- 2、输入还是输出？如果是输出，添加速度值！
- 3、配置低8位
- 4、配置高8位

看别人函数先看框架，可以收起来

```

/*----- GPIO 模式配置 -----*/
// 把输入参数GPIO_Mode的低四位暂存在currentmode
currentmode = ((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x0F);

// bit4是1表示输出，bit4是0则是输入
// 判断bit4是1还是0，即首选判断是输入还是输出模式
if (((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x10)) != 0x00)
{
    // 输出模式则要设置输出速度
    currentmode |= (uint32_t)GPIO_InitStruct->GPIO_Speed;
}

```

currentmode的值为0x00|0x01，即为0x01

STM32F103 GPIO 引脚工作模式真值表分析											
				二进制							
				十六进制							
				bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
					上/下拉		输入/出	具体工作模式 对照寄存器说明			
GPIO_Mode_TypeDef	GPIO_Mode_AIN	模拟输入	0X00	0	0	0	0	0	0	0	0
	GPIO_Mode_IN_FLOATING	浮空输入	0X04	0	0	0	0	0	1	0	0
	GPIO_Mode_IPD	下拉输入	0X28	0	0	1	0	1	0	0	0
	GPIO_Mode_IPU	上拉输入	0X48	0	1	0	0	1	0	0	0
	GPIO_Mode_Out_OD	开漏输出	0X14	0	0	0	1	0	1	0	0
	GPIO_Mode_Out_PP	推挽输出	0X10	0	0	0	1	0	0	0	0
	GPIO_Mode_AF_OD	复用开漏输出	0X1C	0	0	0	1	1	1	0	0
	GPIO_Mode_AF_PP	复用推挽输出	0X18	0	0	0	1	1	0	0	0

这8个宏的高4bit可随意设置，只要能在程序上帮助判断出模式即可，真正写到寄存器的值是bit2和bit3


```

if (((uint32_t)GPIO_InitStruct->GPIO_Pin & ((uint32_t)0x00FF)) != 0x00)
{
// 先备份CRL寄存器的值
tmpreg = GPIOx->CRL;

// 循环, 从Pin0开始配对, 找出具体的Pin
for (pinpos = 0x00; pinpos < 0x08; pinpos++)
{
// pos的值为1左移pinpos位
pos = ((uint32_t)0x01) << pinpos;

// 令pos与输入参数GPIO_PIN作位与运算, 为下面的判断作准备
currentpin = (GPIO_InitStruct->GPIO_Pin) & pos;

// 若currentpin=pos, 则找到使用的引脚
if (currentpin == pos)
{
// pinpos的值左移两位 (乘以4), 因为寄存器中4个寄存器位配置一个引脚
pos = pinpos << 2;
// 把控制这个引脚的4个寄存器位清零, 其它寄存器位不变
pinmask = ((uint32_t)0x0F) << pos;
tmpreg &= ~pinmask;

// 向寄存器写入将要配置的引脚的模式
tmpreg |= (currentmode << pos);

// 判断是否为下拉输入模式
if (GPIO_InitStruct->GPIO_Mode == GPIO_Mode_IPD)
{
// 下拉输入模式, 引脚默认置0, 对BRR寄存器写1可对引脚置0
GPIOx->BRR = (((uint32_t)0x01) << pinpos);
}
else
{
// 判断是否为上拉输入模式
if (GPIO_InitStruct->GPIO_Mode == GPIO_Mode_IPU)
{
// 上拉输入模式, 引脚默认值为1, 对BSRR寄存器写1可对引脚置1
GPIOx->BSRR = (((uint32_t)0x01) << pinpos);
}
}
}
}
// 把前面处理后的暂存值写入到CRL寄存器之中
GPIOx->CRL = tmpreg;
}

```

```

GPIO_InitStructure.GPIO_Pin = 0b0010 0000;
GPIO_InitStructure.GPIO_Mode = 0b0001 0000;
GPIO_InitStructure.GPIO_Speed = 0b0000 0001;
GPIO_Init(GPIOB, &GPIO_InitStructure);

```

进入for语句第1次循环, 此时pinpos为0

pos = 0x01 << 0 pos = 0x01

currentpin的值为0b0010 0000 & 0x01 则currentpin为0

进入for语句第2次循环, 此时pinpos为1

pos = 0x01 << 1 pos = 0x02 (0b0000 0010)

currentpin的值为0b0010 0000 & 0x02 则currentpin为0

进入for语句第6次循环, 此时pinpos为5

pos = 0x01 << 5 pos = 0x20 (0b0010 0000)

currentpin的值为0b0010 0000 & 0x20 则currentpin为0x20

```

if (((uint32_t)GPIO_InitStruct->GPIO_Pin & ((uint32_t)0x00FF)) != 0x00)
{
// 先备份CRL寄存器的值
tmpreg = GPIOx->CRL;

// 循环, 从Pin0开始配对, 找出具体的Pin
for (pinpos = 0x00; pinpos < 0x08; pinpos++)
{
// pos的值为1左移pinpos位
pos = ((uint32_t)0x01) << pinpos;

// 令pos与输入参数GPIO_PIN作位与运算, 为下面的判断作准备
currentpin = (GPIO_InitStruct->GPIO_Pin) & pos;

//若currentpin=pos, 则找到使用的引脚
if (currentpin == pos)
{
// pinpos的值左移两位 (乘以4), 因为寄存器中4个寄存器位配置一个引脚
pos = pinpos << 2;
//把控制这个引脚的4个寄存器位清零, 其它寄存器位不变
pinmask = ((uint32_t)0x0F) << pos;
tmpreg &= ~pinmask;

// 向寄存器写入将要配置的引脚的模式
tmpreg |= (currentmode << pos);

// 判断是否为下拉输入模式
if (GPIO_InitStruct->GPIO_Mode == GPIO_Mode_IPD)
{
// 下拉输入模式, 引脚默认置0, 对BRR寄存器写1可对引脚置0
GPIOx->BRR = (((uint32_t)0x01) << pinpos);
}
else
{
// 判断是否为上拉输入模式
if (GPIO_InitStruct->GPIO_Mode == GPIO_Mode_IPU)
{
// 上拉输入模式, 引脚默认值为1, 对BSRR寄存器写1可对引脚置1
GPIOx->BSRR = (((uint32_t)0x01) << pinpos);
}
}
}
}

// 把前面处理后的暂存值写入到CRL寄存器之中
GPIOx->CRL = tmpreg;
}

```

```

GPIO_InitStructure.GPIO_Pin = 0b0010 0000;
GPIO_InitStructure.GPIO_Mode = 0b0001 0000;
GPIO_InitStructure.GPIO_Speed = 0b0000 0001;
GPIO_Init(GPIOB, &GPIO_InitStructure);

```

进入for语句第6次循环, 此时pinpos为5

$pos = 0x01 \ll 5$ $pos = 0x20$ (0b0010 0000)

currentpin的值为0b0010 0000&0x20 则currentpin为0x20

$pos = pinpos \ll 2$ pos的值为具体的引脚值*4

$pos = pinpos \ll 2 = 5 \ll 2 = 20$

$pinmask = 0x0F \ll 20 = 0b0000$ 1111 << 20
 $= 0b0000$ 0000 1111 0000 0000 0000 0000 0000

$tmpreg = tmpreg \& \sim pinmask =$
 $= tmpreg \& 1111$ 1111 0000 1111 1111 1111 1111 1111

$tmpreg \mid = currentmode \ll pos$ $currentmode = 0x01$

$tmpreg \mid = 0x01 \ll 20 = 0b0001$ 0000 0000 0000 0000 0000

把tmpreg 的值写入CRL寄存器

void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)

```
currentmode=0x0000;  
currentmode=0x0001;  
tmpreg = GPIOx->CRL  
for to find currentpos=0x20 ; pinpos=5 ;  
pos=20 ;  
pinmask=0x00F00000 ;  
tmpreg &=~0xFF0FFFFFFF ;  
tmpreg |= 0x0001<<20 ;  
GPIOx->CRL = tmpreg;
```

```
GPIOB->CRL &= 0xFF0FFFFFFF;  
GPIOB->CRL |= 0x00100000;
```

```
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5; //0x0020;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //0x0010;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz; //0x0001  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

STM32F103 GPIO 引脚工作模式真值表分析											
			十六进制	二进制							
				bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
				上/下拉		输入/出		具体工作模式 对照寄存器说明			
GPIO_Mode_TypeDef	GPIO_Mode_AIN	模拟输入	0X00	0	0	0	0	0	0	0	0
	GPIO_Mode_IN_FLOATING	浮空输入	0X04	0	0	0	0	0	1	0	0
	GPIO_Mode_IPD	下拉输入	0X28	0	0	1	0	1	0	0	0
	GPIO_Mode_IPU	上拉输入	0X48	0	1	0	0	1	0	0	0
	GPIO_Mode_Out_OD	开漏输出	0X14	0	0	0	1	0	1	0	0
	GPIO_Mode_Out_PP	推挽输出	0X10	0	0	0	1	0	0	0	0
	GPIO_Mode_AF_OD	复用开漏输出	0X1C	0	0	0	1	1	1	0	0
	GPIO_Mode_AF_PP	复用推挽输出	0X18	0	0	0	1	1	0	0	0
这8个宏的高4bit可随意设置，只要能在程序上帮助判断出模式即可，真正写到寄存器的值是bit2和bit3											

库函数最终操作的对象也是寄存器。

9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

0b0001 0000 0000 0000 0000 0000

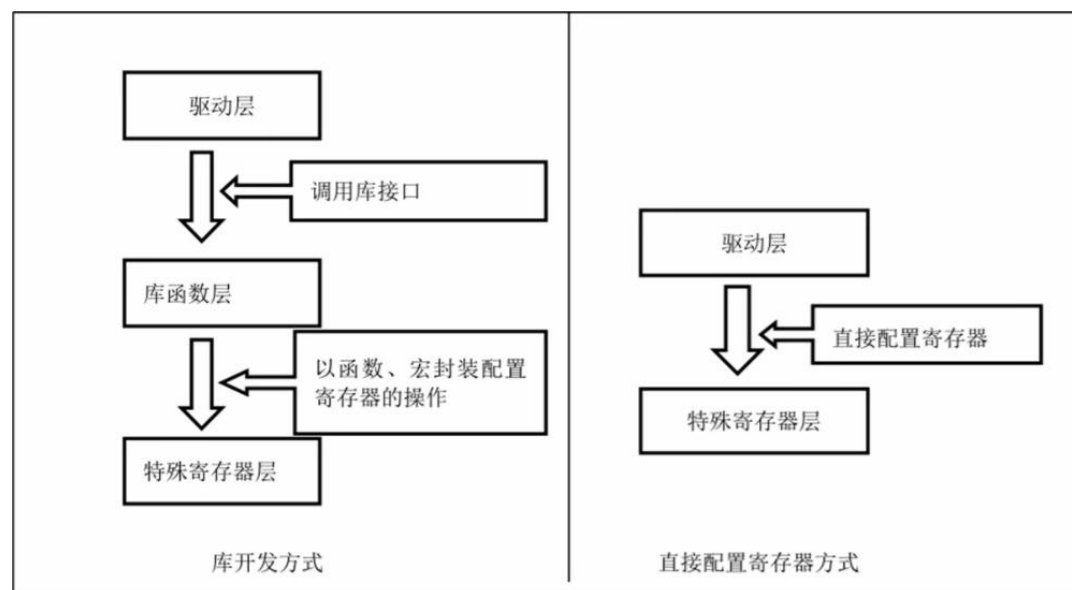
```
//配置IO口PB5为输出,推挽输出,速率为10M
GPIOB_CRL&=0xFF0FFFFFF;
GPIOB_CRL|=0X00100000;
```


Part II : Firmware Library

What's and Why FWLib

Firmware Library is a collection of functions, and the functions are packages of statements that control registers. When the corresponding actions need to be performed, just call the corresponding functions and configure the parameters ;

Firmware Library  Building block



FWLib: Good transplantation; High readability
Register: More efficient

Get the FWLib

STM32 Standard Peripheral Libraries Save to MyST

[Overview](#) [Product selector](#) [Documentation](#)

STM32 Standard Peripheral Libraries

STM32 Standard Peripheral Libraries Expansion

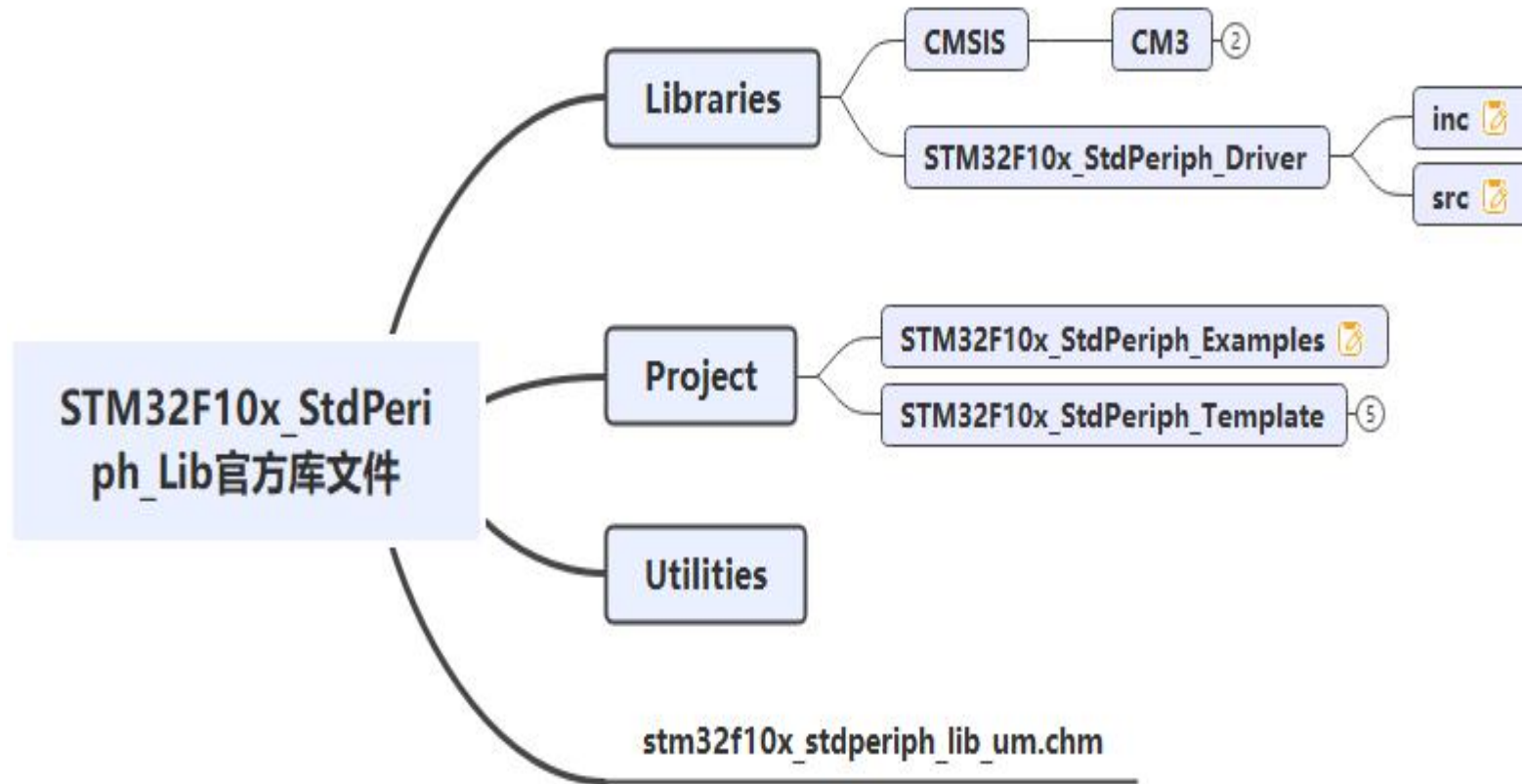
STM32 Standard Peripheral Drivers

F0 F1 F3 F3 F2 F4 L1

STM32

STM32F37x/38x
STM32F301x/302x/303x/334x

<https://www.st.com/zh/embedded-software/stsw-stm32054.html>



How to configure those files?

Background of the FWLib

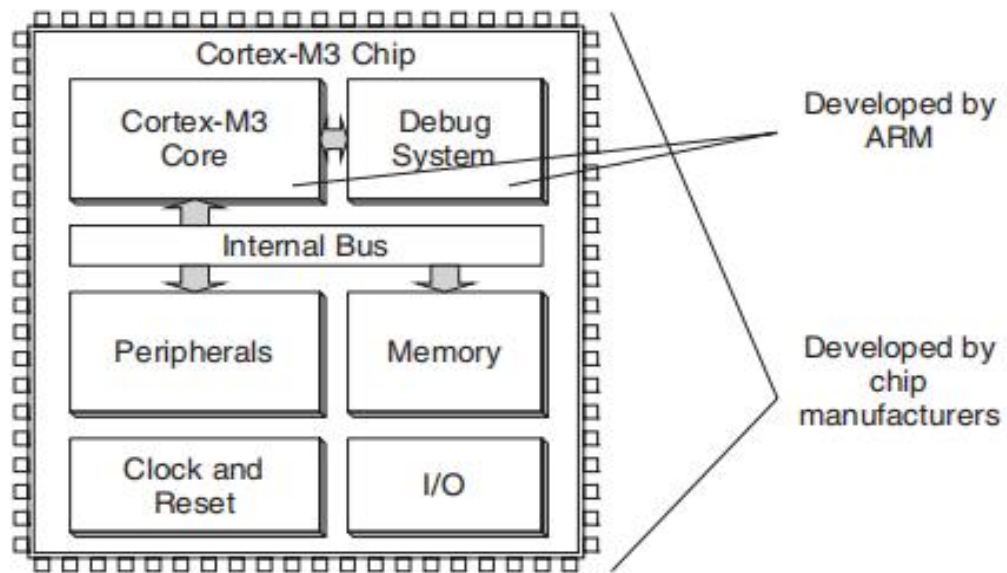


Figure 1.1 The Cortex-M3 Processor vs the Cortex-M3-Based MCU

ARM: Chip Core、IP

TI , ST , AD : Chip Manufacturers, design components outside the core, which is called **peripherals.(most objects we programming、 control)**

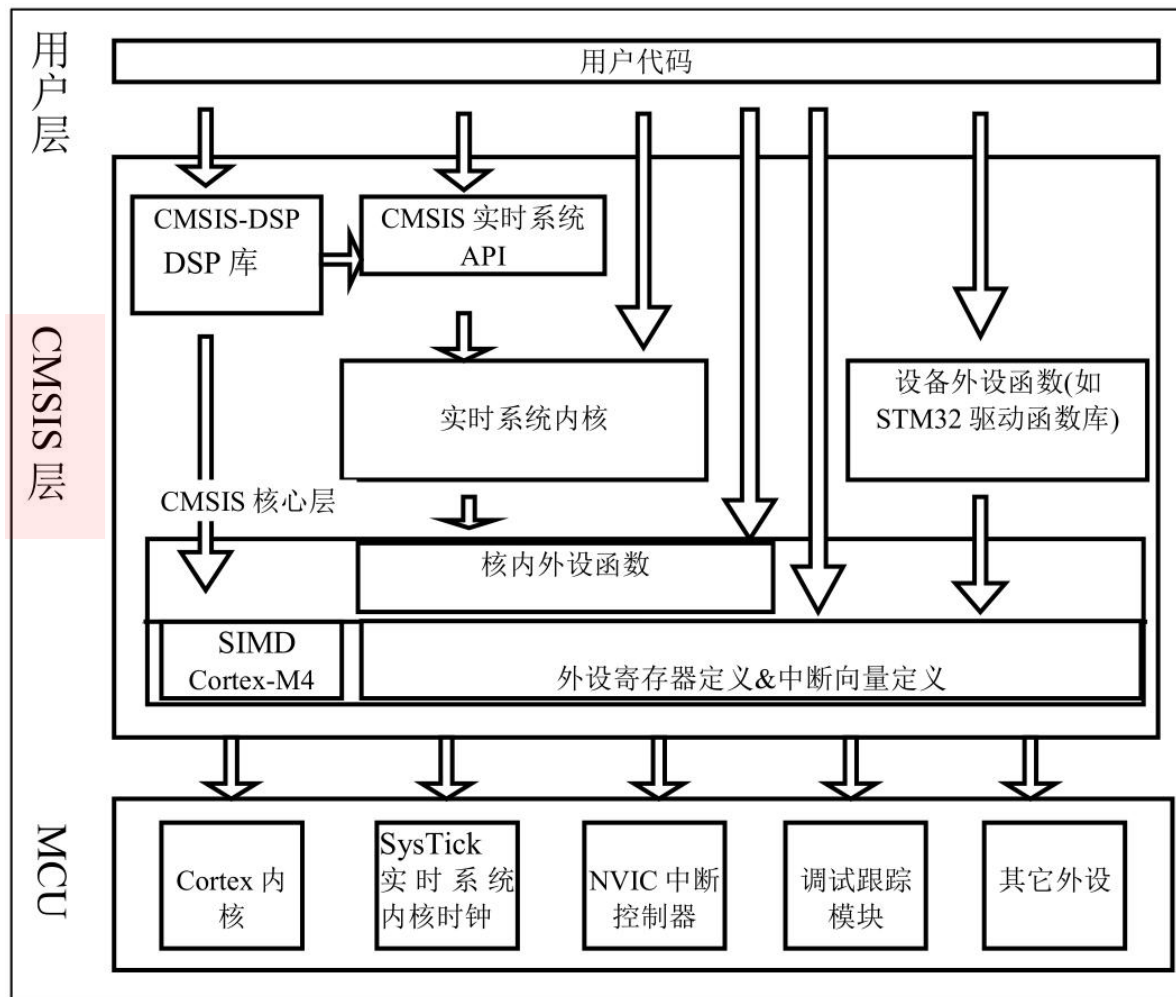
Chips with Cortex-M3, the same core, different peripherals.

→ different control methods

`GPIO_Init();`

`GPIO_Init_ST();`

ARM and the Chip Manufacturers established the CMSIS
(Cortex Microcontroller Software Interface Standard)



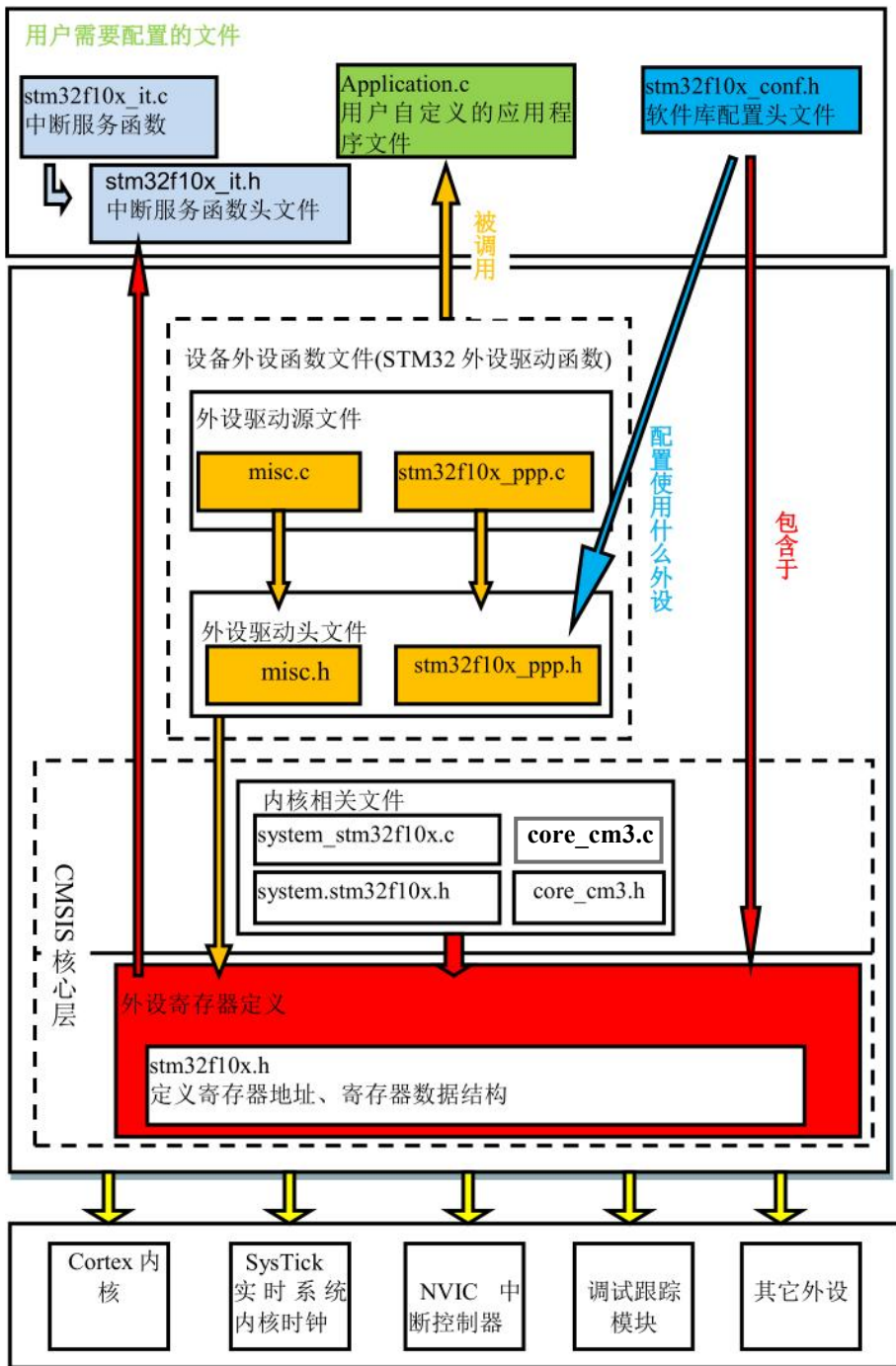
CMSIS Architecture

CMSIS layer: most important, in the middle layer.
Upward: provides the function interface required by the user layer
Downward: provides the functions communicating with the core and other peripherals

用户层

CMSIS层

MCU层



Configuring FWLib ↔ Configuring these function files correctly

stm32f10x_it.c stm32f10x_it.h stm32f10x_conf.h

Applicationxxx.c Applicationxxx.h

USER

stm32f10x_ppp.c stm32f10x_ppp.h (ppp is name of peripherals)

misc.c misc.h(NVIC , SysTick)

system_stm32f10x.c system_stm32f10x.h

core_cm3.c core_cm3.h

CMSIS

stm32f10x.h

startup_stm32f10x_xx.s



STM32F10x Standard Peripherals Firmware Library

隐藏 查找 上一步 前进 停止 刷新 主页 字体 打印 选项(O)

目录(C) 索引(N) 搜索(S) 收藏夹(I)

STM32F10x Standard Peripherals Library

Related Pages

STM32F10x Standard Peripheral's Drivers

STM32F10x Standard Peripheral's Examples

- 3 ADCs DMA example
- ADC1 DMA example
- ADC analog watchdog example
- ADC external lines trigger example
- ADC regular simultaneous dual mode example
- ADC TIM trigger and auto-injection example
- Backup Data example
- BKP Tamper example
- Dual CAN example
- CAN LoopBack example
- CAN Networking example
- CEC Data Exchange using Interrupt example
- CortexM3 BitBand example
- CortexM3 Mode Privilege example
- CortexM3 MPU example
- CRC Calculation example
- DAC dual mode sine wave example
- DAC one channel noise wave example
- DAC one channel DMA escalator example
- DAC two channels triangle wave example

DMA_EXAMPLES

- DMA ADC1 TIM1 example
- DMA FLASH to RAM example
- DMA FSMC example
- DMA I2C to RAM example
- DMA SPI to RAM example
- EXTI configuration example
- FLASH Program example
- XL-Density devices FLASH Dual Boot capability
- FLASH Write Protection example
- FSMC NAND example
- FSMC NOR example

src

- [misc.c](#)
- [stm32f10x_adc.c](#)
- [stm32f10x_bkp.c](#)
- [stm32f10x_can.c](#)
- [stm32f10x_cec.c](#)
- [stm32f10x_crc.c](#)
- [stm32f10x_dac.c](#)
- [stm32f10x_dbgmcu.c](#)
- [stm32f10x_dma.c](#)
- [stm32f10x_exti.c](#)
- [stm32f10x_flash.c](#)
- [stm32f10x_fsmc.c](#)
- [stm32f10x_gpio.c](#)
- [stm32f10x_i2c.c](#)
- [stm32f10x_iwdg.c](#)
- [stm32f10x_pwr.c](#)
- [stm32f10x_rcc.c](#)
- [stm32f10x_rtc.c](#)
- [stm32f10x_sdio.c](#)
- [stm32f10x_spi.c](#)
- [stm32f10x_tim.c](#)
- [stm32f10x_usart.c](#)
- [stm32f10x_iwdg.c](#)

inc

- [misc.h](#)
- [stm32f10x_adc.h](#)
- [stm32f10x_bkp.h](#)
- [stm32f10x_can.h](#)
- [stm32f10x_cec.h](#)
- [stm32f10x_crc.h](#)
- [stm32f10x_dac.h](#)
- [stm32f10x_dbgmcu.h](#)
- [stm32f10x_dma.h](#)
- [stm32f10x_exti.h](#)
- [stm32f10x_flash.h](#)
- [stm32f10x_fsmc.h](#)
- [stm32f10x_gpio.h](#)
- [stm32f10x_i2c.h](#)
- [stm32f10x_iwdg.h](#)
- [stm32f10x_pwr.h](#)
- [stm32f10x_rcc.h](#)
- [stm32f10x_rtc.h](#)
- [stm32f10x_sdio.h](#)
- [stm32f10x_spi.h](#)
- [stm32f10x_tim.h](#)
- [stm32f10x_usart.h](#)

Help Document

Initializes the GPIOx peripheral according to the specified parameters in the GPIO_InitStruct.

GPIOx_: where x can be (A..G) to select the GPIO peripheral.
GPIO_InitStruct_: pointer to a **GPIO_InitTypeDef** structure that contains the configuration information for the specified GPIO peripheral.

None

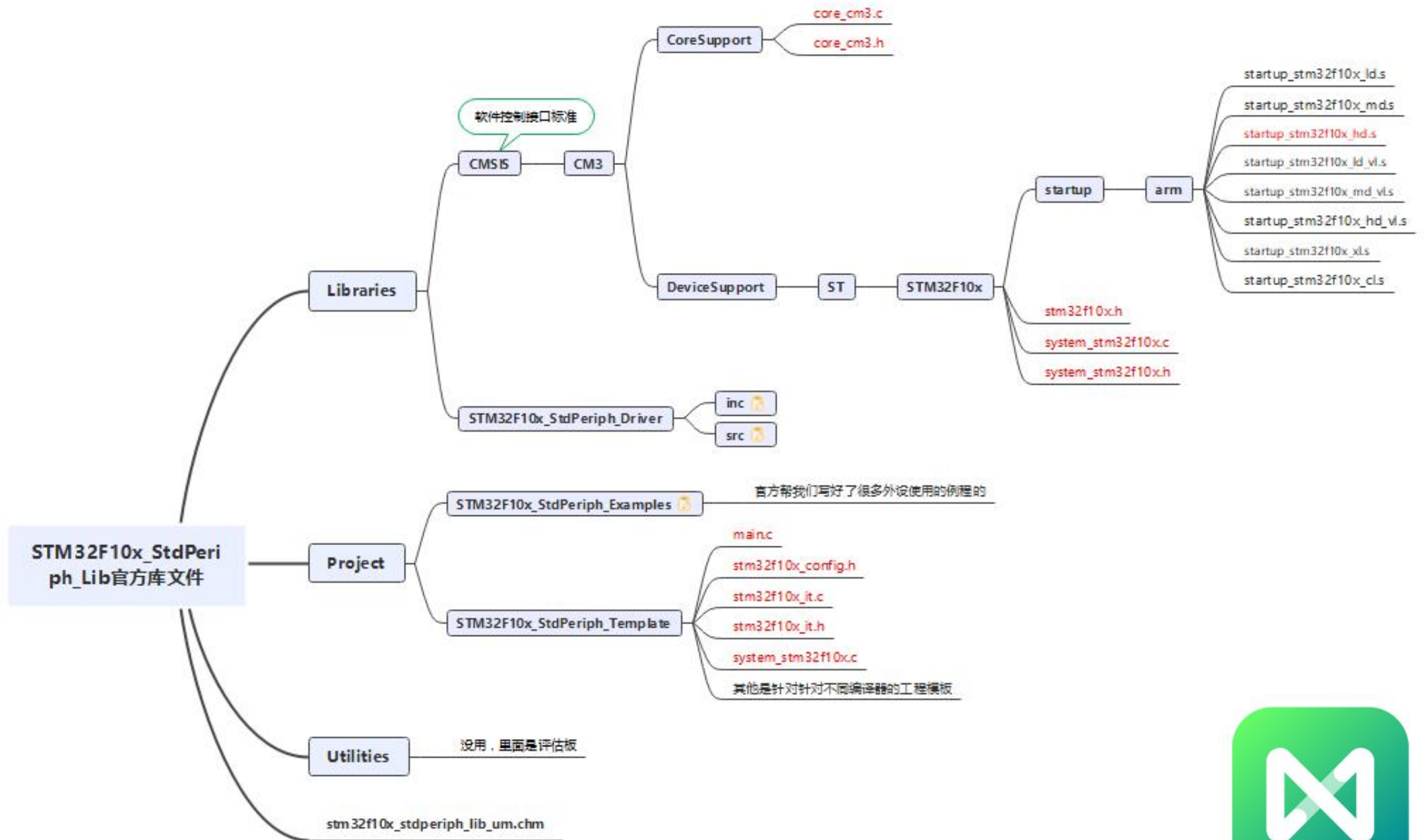
Definition at line 172 of file stm32f10x_gpio.c.

References [assert_param](#), [GPIO_TypeDef::BRR](#), [GPIO_TypeDef::BSRR](#), [GPIO_TypeDef::CRH](#), [GPIO_TypeDef::CRL](#), [GPIO_InitTypeDef::GPIO Mode](#), [GPIO_InitTypeDef::GPIO Pin](#), [GPIO_InitTypeDef::GPIO Speed](#), [IS_GPIO_ALL_PERIPH](#), [IS_GPIO_MODE](#), [IS_GPIO_MODE](#), [IS_GPIO_MODE](#)

Referenced by [CAN Config\(\)](#), [EXTI0 Config\(\)](#), [EXTI9_5 Config\(\)](#), [GPIO Configuration\(\)](#), [HDMI CEC Init\(\)](#), [IOE EXTI Config\(\)](#), [IOE EXTI DeInit\(\)](#), [LCD DeInit\(\)](#), [LCD GPIOConfig\(\)](#), [LCD SPIConfig\(\)](#), [LM75 LowLevel DeInit\(\)](#), [LM75 LowLevel Init\(\)](#), [main\(\)](#), [NAND Init\(\)](#), [NOE Init\(\)](#), [SD LowLevel Init\(\)](#), [sEE LowLevel DeInit\(\)](#), [sEE LowLevel Init\(\)](#), [sFLASH LowLevel DeInit\(\)](#), [sFLASH LowLevel Init\(\)](#), [SRA Init\(\)](#), [STM EVAL COMInit\(\)](#), [STM EVAL LEDInit\(\)](#), and [STM EVAL PBInit\(\)](#).

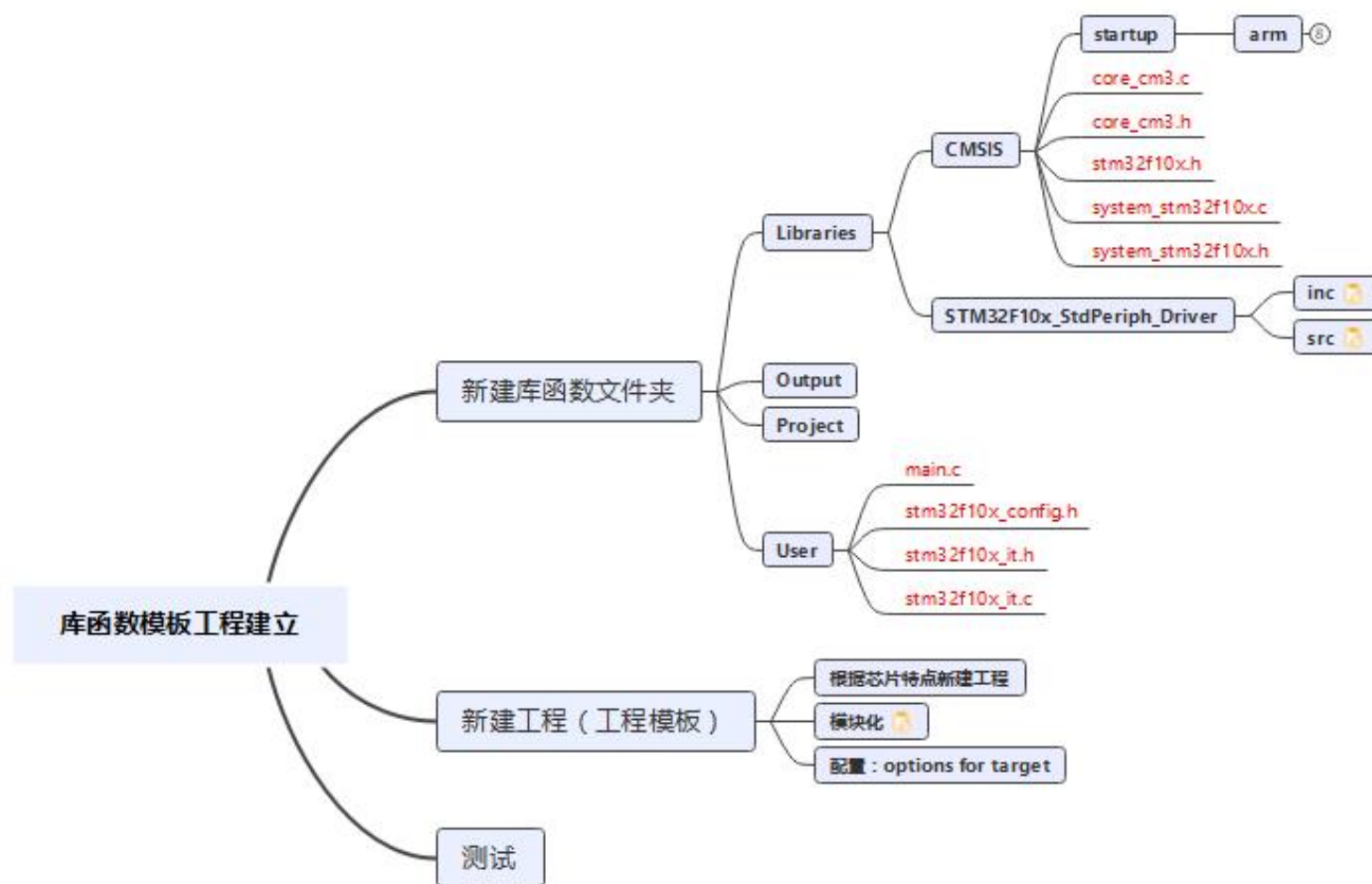
Analysis of the FWLib

名称	修改日期	类型	大小
 _htmresc	2021/10/8 22:27	文件夹	
 Libraries	2021/10/8 22:27	文件夹	
 Project	2021/10/8 22:27	文件夹	
 Utilities	2021/10/8 22:27	文件夹	
 Package_license.html	2021/10/8 22:22	Chrome HTML D...	81 KB
 Package_license.md	2021/10/8 22:23	Markdown	15 KB
 Release_Notes.html	2021/10/8 22:23	Chrome HTML D...	131 KB
 stm32f10x_stdperiph_lib_um.chm	2021/10/8 22:23	编译的 HTML 帮...	19,321 KB



Part III : My Template_FWLib_Project

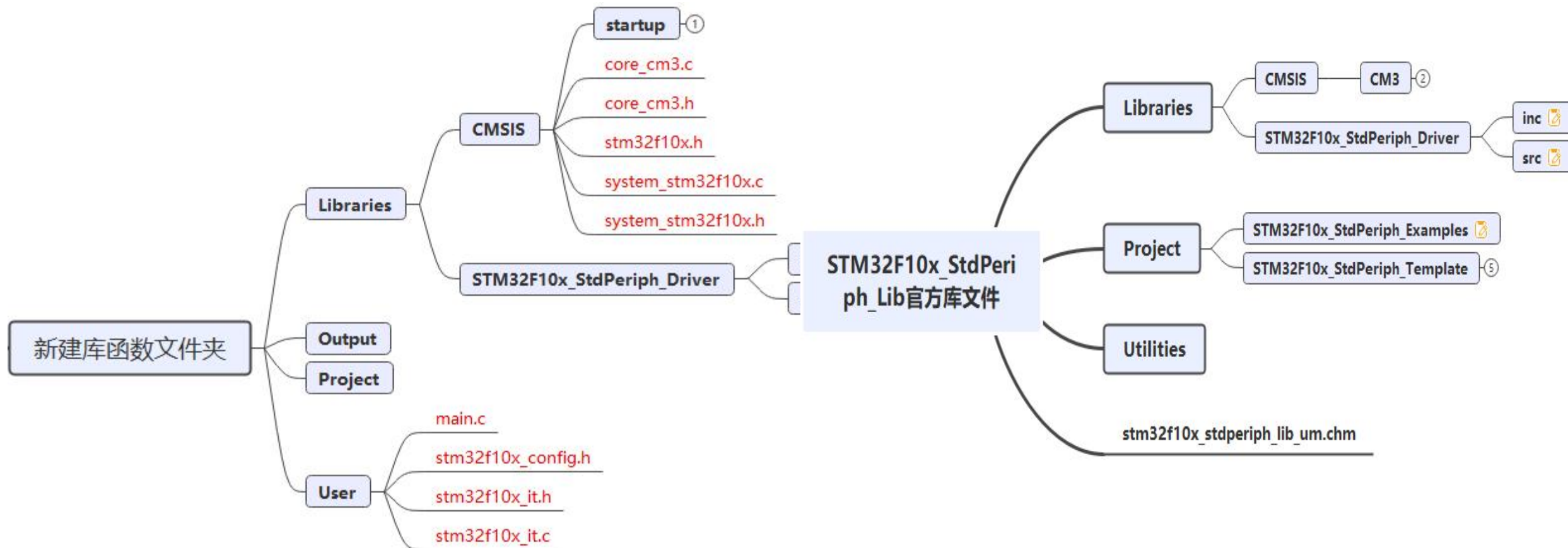
My Template_FWLib_Project



Practical Demonstration

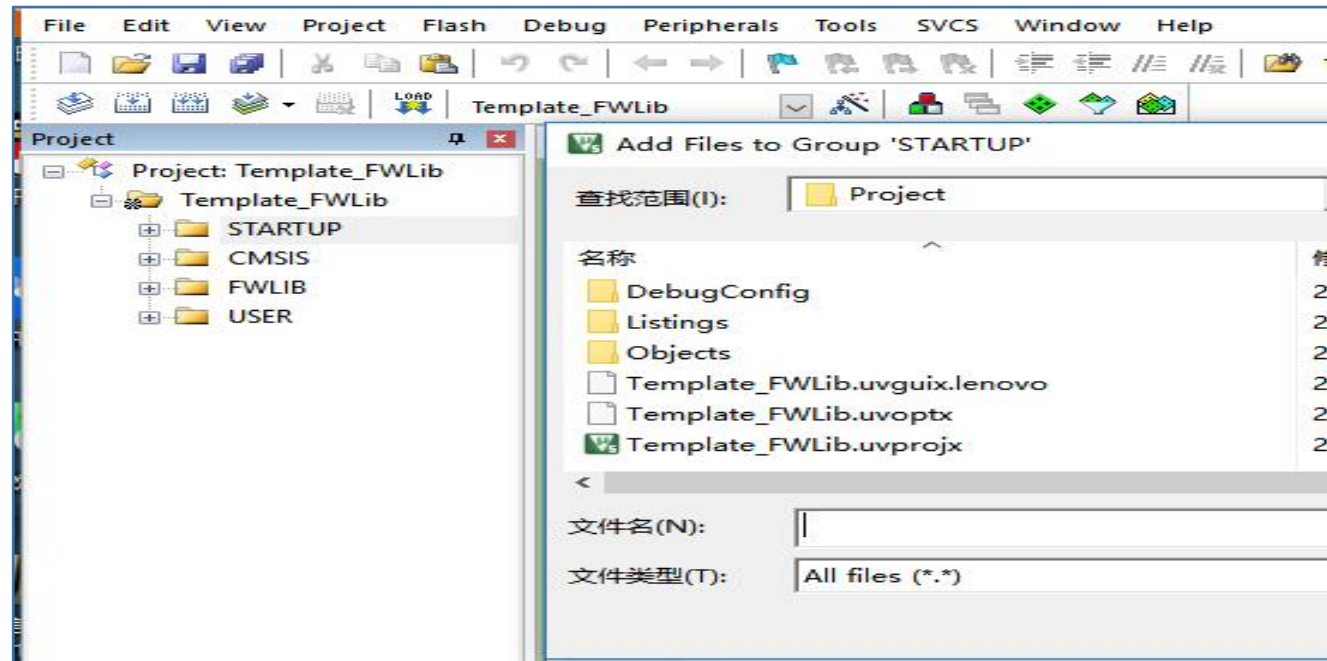
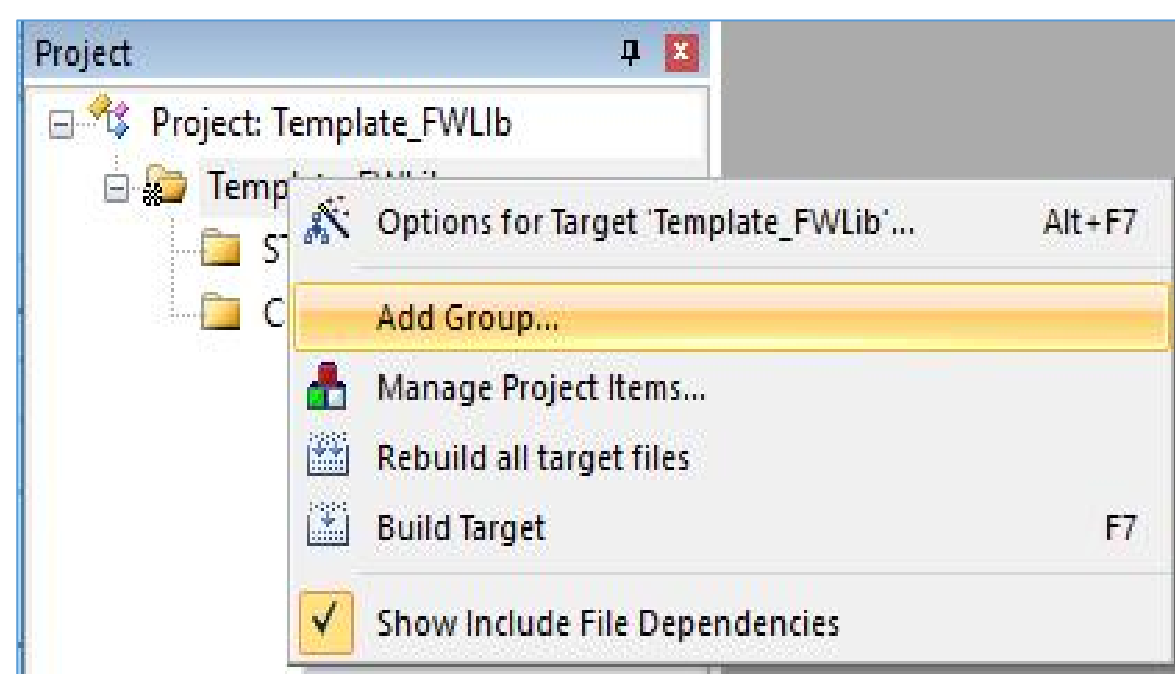
Template Folder

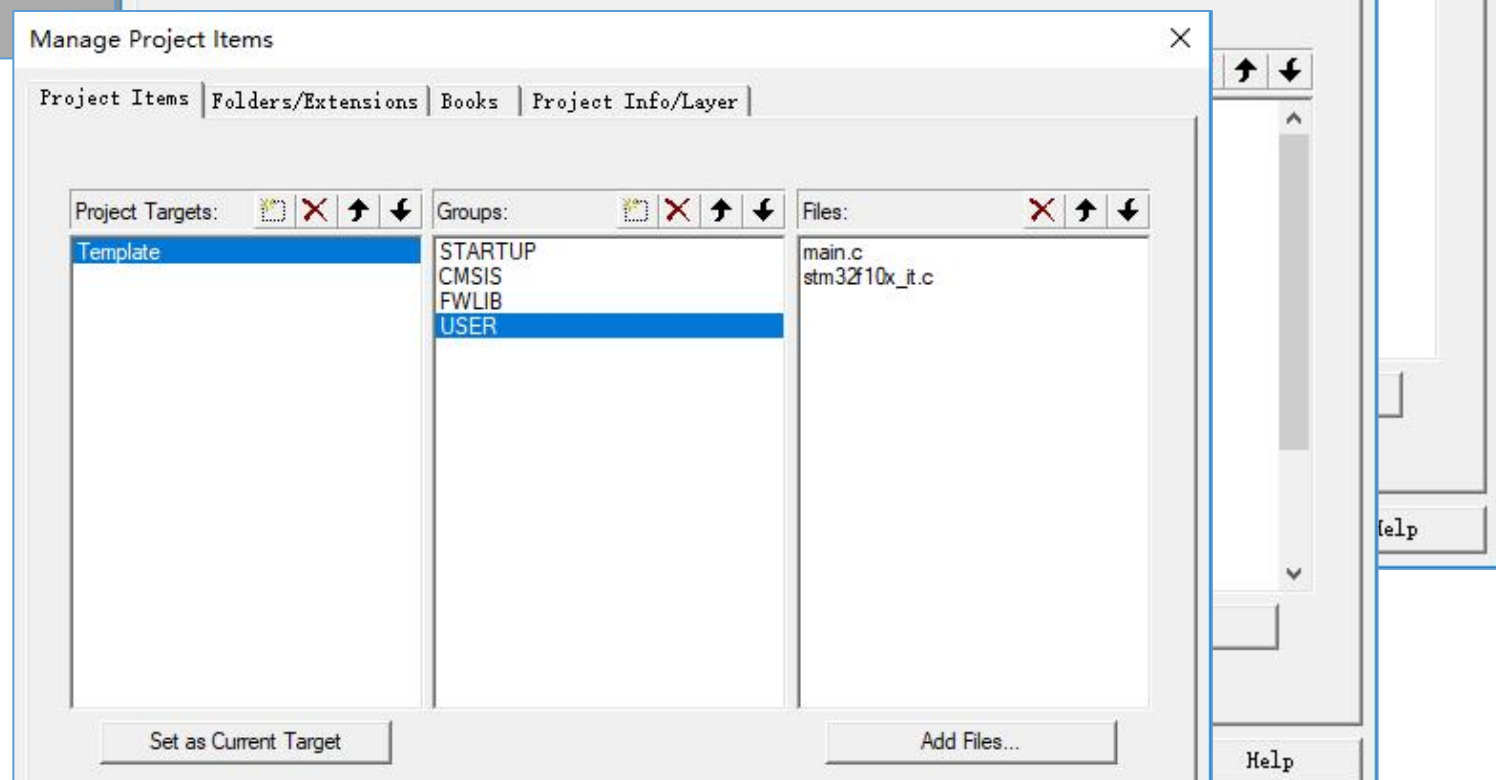
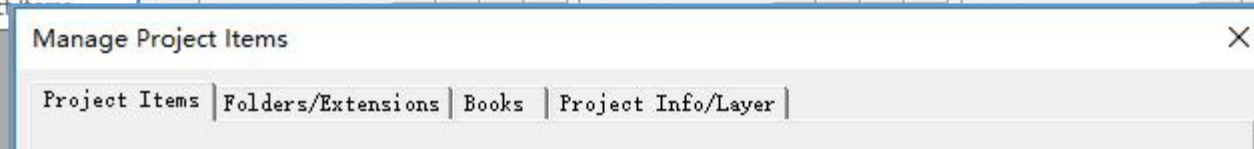
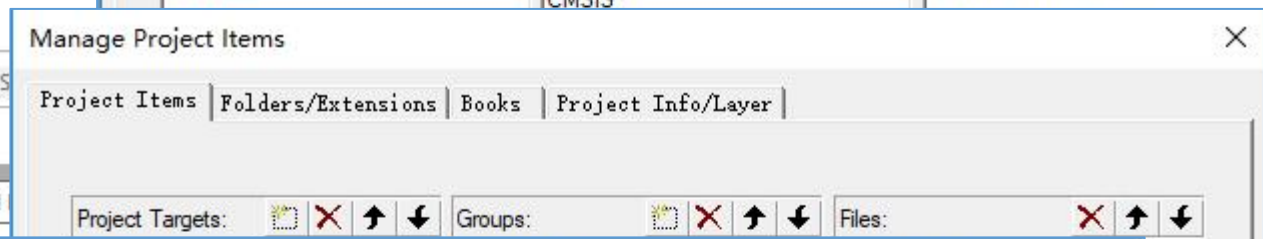
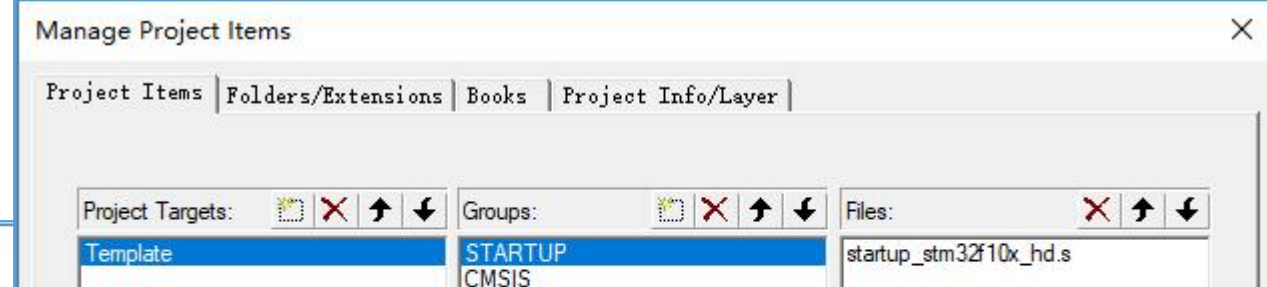
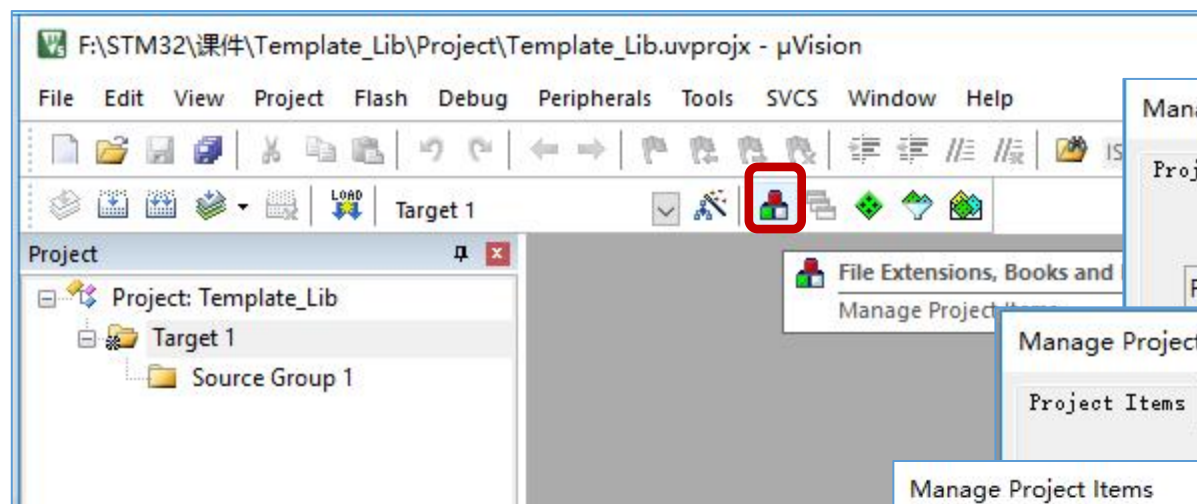
Template_FWLib

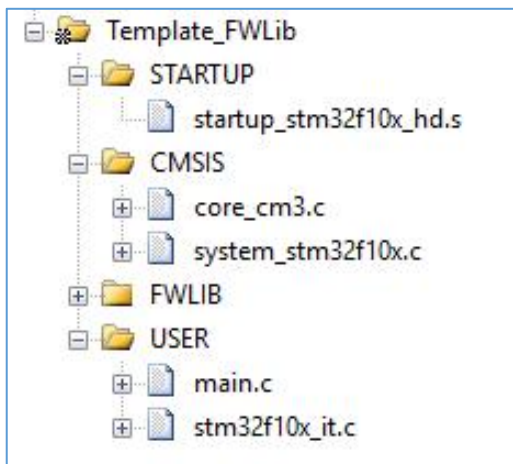


New Template_FWLib_Project

- step1: new project-->save in ../Template_FWLib\Project --> select CPU
- step2: Add group folder
- step3: Add library files manually: double-click folder/ 

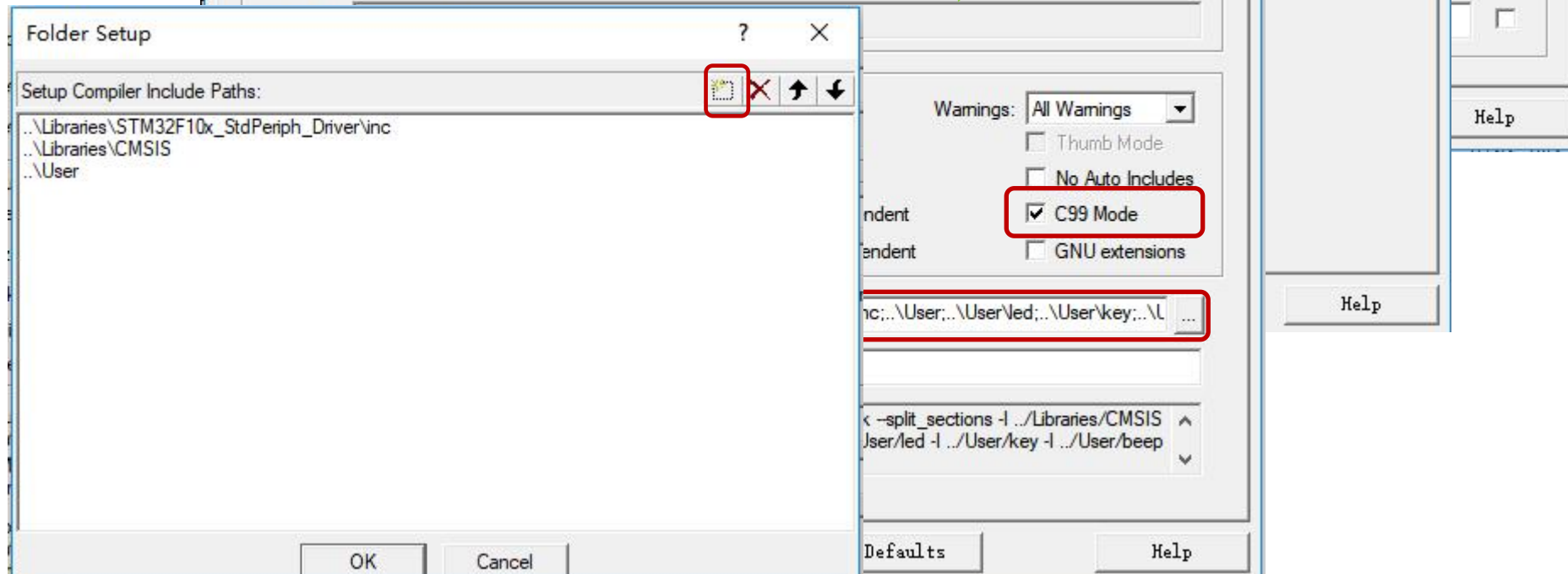
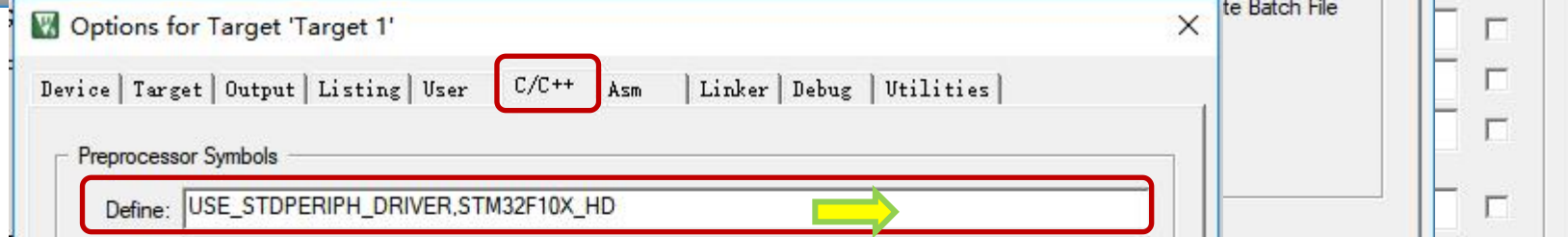
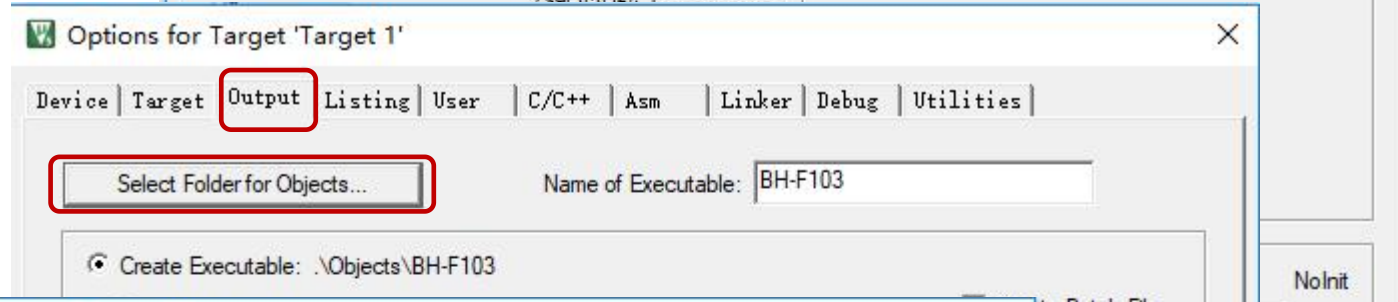
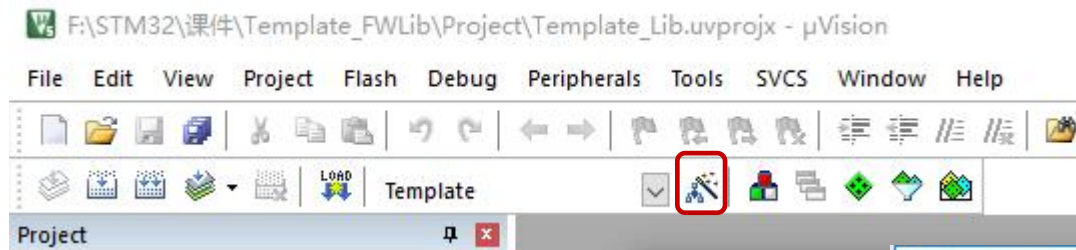






Folder Name	Filese
STARTUP	startup_stm32f10x_hd.s
CMSIS	core_cm3.c system_stm32f10x.c
FWLIB	all .c files in STM32F10x_StdPeriph_Driver\src , this is just Firware
USER	main.c (empty now) stm32f10x_it.c(interrupt file, empty now)

Configuration



main.c

```
#include "stm32f10x.h" //

int main(void)
{
}
```

Build(F7)

---->

```
"..\Output\Template.axf" - 0 Error(s), 0 Warning(s).
```

Template_FWLib_Project is Completed now !!!

Test

main.c

```
#include "stm32f10x.h" //
int main(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    // 打开 GPIOB 端口的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    // 配置IO口为输出
    GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    // 控制 ODR 寄存器
    GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
    //GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
}
```



What's RCC ?

How did I know there's such a function ?

Where should we find this function?

How to configure the parameters ?

Download configuration (Debug) --->Download --> Check

From now on, we programme with the Firmware Library ;

But remember: The **essence** of Firmware Library Programming is actually the **configuration of registers**-----The truth just is: others have written some commonly used functions for us. We only to know where to call the function and how to configure the parameters now.

Homework of lecture1

- Install the softwares on your computer
- Realize the lighting experiment of LED
- Write a summary report:
 - ✓ The introduction to STM32
 - ✓ complete the question in the class(modify LED to PC13)

Homework of lecture2

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```



```
GPIOB->CRL &= 0xFF0FFFFFF;  
GPIOB->CRL |= 0x00100000;
```

Firmware library programming

Register Programming

Question

Configure PC13 as a pull-up input pin by 2 method refer to the example above ,

1 : Firmware library programming

2 : Register Programming

Homework of lecture3

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_2;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPD ;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

库函数最终操作的对象也是寄存器。

- ✓ Write out the assignment process of each step in the function `GPIO_Init(GPIOB, &GPIO_InitStructure)`
- ✓ Sort out the file structure of the official Firmware Library as page27 (mind mapping is recommended).

Suggestions about how to learn STM32

- Read those manuals More,More,More.....
- Practice more,no matter how simple the program is,
Practice it by yourself !
- Take more notes!!

USE_STDPERIPH_DRIVER, STM32F10X_HD

 misc.c	misc.h
 stm32f10x_adc.c	stm32f10x_adc.h
 stm32f10x_bkp.c	stm32f10x_bkp.h
 stm32f10x_can.c	stm32f10x_can.h
 stm32f10x_cec.c	stm32f10x_cec.h
 stm32f10x_crc.c	stm32f10x_crc.h
 stm32f10x_dac.c	stm32f10x_dac.h
 stm32f10x_dbgmcu.c	stm32f10x_dbgmcu.h
 stm32f10x_dma.c	stm32f10x_dma.h
 stm32f10x_exti.c	stm32f10x_exti.h
 stm32f10x_flash.c	stm32f10x_flash.h
 stm32f10x_fsmc.c	stm32f10x_fsmc.h
 stm32f10x_gpio.c	stm32f10x_gpio.h
 stm32f10x_i2c.c	stm32f10x_i2c.h
 stm32f10x_iwdg.c	stm32f10x_iwdg.h
 stm32f10x_pwr.c	stm32f10x_pwr.h
 stm32f10x_rcc.c	stm32f10x_rcc.h
 stm32f10x_rtc.c	stm32f10x_rtc.h
 stm32f10x_sdio.c	stm32f10x_sdio.h
 stm32f10x_spi.c	stm32f10x_spi.h
 stm32f10x_tim.c	stm32f10x_tim.h
 stm32f10x_usart.c	stm32f10x_usart.h
 stm32f10x_wwdg.c	stm32f10x_wwdg.h

stm32f10x_conf.h

```
stm32f10x_conf.h
22  /* Define to prevent recursive in
23  #ifndef __STM32F10X_CONF_H
24  #define __STM32F10X_CONF_H
25
26  /* Includes -----
27  /* Uncomment/Comment the line bel
28  #include "stm32f10x_adc.h"
29  #include "stm32f10x_bkp.h"
30  #include "stm32f10x_can.h"
31  #include "stm32f10x_cec.h"
32  #include "stm32f10x_crc.h"
33  #include "stm32f10x_dac.h"
34  #include "stm32f10x_dbgmcu.h"
35  #include "stm32f10x_dma.h"
36  #include "stm32f10x_exti.h"
37  #include "stm32f10x_flash.h"
38  #include "stm32f10x_fsmc.h"
39  #include "stm32f10x_gpio.h"
40  #include "stm32f10x_i2c.h"
41  #include "stm32f10x_iwdg.h"
42  #include "stm32f10x_pwr.h"
43  #include "stm32f10x_rcc.h"
44  #include "stm32f10x_rtc.h"
45  #include "stm32f10x_sdio.h"
46  #include "stm32f10x_spi.h"
47  #include "stm32f10x_tim.h"
48  #include "stm32f10x_usart.h"
49  #include "stm32f10x_wwdg.h"
50  #include "misc.h" /* High level f
```

stm32f10x.h

```
stm32f10x.h
8294  */
8295
8296  #ifdef USE_STDPERIPH_DRIVER
8297  #include "stm32f10x_conf.h"
8298  #endif
```

USE STDPERIPH DRIVER,STM32F10X_HD

```
stm32f10x.h
297 #ifndef STM32F10X_HD
298     ADC1_2_IRQn          = 18,    /*< ADC1 and ADC2 global Interrupt */
299     USB_HP_CAN1_TX_IRQn  = 19,    /*< USB Device High Priority or CAN1 TX Interrupts */
300     USB_LP_CAN1_RX0_IRQn = 20,    /*< USB Device Low Priority or CAN1 RX0 Interrupts */
301     CAN1_RX1_IRQn        = 21,    /*< CAN1 RX1 Interrupt */
302     CAN1_SCE_IRQn        = 22,    /*< CAN1 SCE Interrupt */
303     EXTI9_5_IRQn         = 23,    /*< External Line[9:5] Interrupts */
304     TIM1_BRK_IRQn        = 24,    /*< TIM1 Break Interrupt */
305     TIM1_UP_IRQn         = 25,    /*< TIM1 Update Interrupt */
306     TIM1_TRG_COM_IRQn    = 26,    /*< TIM1 Trigger and Commutation Interrupt */
307     TIM1_CC_IRQn         = 27,    /*< TIM1 Capture Compare Interrupt */
308     TIM2_IRQn            = 28,    /*< TIM2 global Interrupt */
309     TIM3_IRQn            = 29,    /*< TIM3 global Interrupt */
310     TIM4_IRQn            = 30,    /*< TIM4 global Interrupt */
311     I2C1_EV_IRQn         = 31,    /*< I2C1 Event Interrupt */
312     I2C1_ER_IRQn         = 32,    /*< I2C1 Error Interrupt */
313     I2C2_EV_IRQn         = 33,    /*< I2C2 Event Interrupt */
314     I2C2_ER_IRQn         = 34,    /*< I2C2 Error Interrupt */
315     SPI1_IRQn            = 35,    /*< SPI1 global Interrupt */
316     SPI2_IRQn            = 36,    /*< SPI2 global Interrupt */
317     USART1_IRQn          = 37,    /*< USART1 global Interrupt */
318     USART2_IRQn          = 38,    /*< USART2 global Interrupt */
319     USART3_IRQn          = 39,    /*< USART3 global Interrupt */
320     EXTI15_10_IRQn       = 40,    /*< External Line[15:10] Interrupts */
321     RTCAlarm_IRQn        = 41,    /*< RTC Alarm through EXTI Line Interrupt */
322     USBWakeUp_IRQn       = 42,    /*< USB Device WakeUp from suspend through EXTI Line Interrupt */
323     TIM8_BRK_IRQn        = 43,    /*< TIM8 Break Interrupt */
324     TIM8_UP_IRQn         = 44,    /*< TIM8 Update Interrupt */
325     TIM8_TRG_COM_IRQn    = 45,    /*< TIM8 Trigger and Commutation Interrupt */
326     TIM8_CC_IRQn         = 46,    /*< TIM8 Capture Compare Interrupt */
327     ADC3_IRQn            = 47,    /*< ADC3 global Interrupt */
328     FSMC_IRQn            = 48,    /*< FSMC global Interrupt */
329     SDIO_IRQn            = 49,    /*< SDIO global Interrupt */
330     TIM5_IRQn            = 50,    /*< TIM5 global Interrupt */
331     SPI3_IRQn            = 51,    /*< SPI3 global Interrupt */
332     UART4_IRQn           = 52,    /*< UART4 global Interrupt */
333     UART5_IRQn           = 53,    /*< UART5 global Interrupt */
334     TIM6_IRQn            = 54,    /*< TIM6 global Interrupt */
335     TIM7_IRQn            = 55,    /*< TIM7 global Interrupt */
336     DMA2_Channel1_IRQn   = 56,    /*< DMA2 Channel 1 global Interrupt */
337     DMA2_Channel2_IRQn   = 57,    /*< DMA2 Channel 2 global Interrupt */
338     DMA2_Channel3_IRQn   = 58,    /*< DMA2 Channel 3 global Interrupt */
339     DMA2_Channel4_5_IRQn = 59,    /*< DMA2 Channel 4 and Channel 5 global Interrupt */
340 #endif /* STM32F10X_HD */
```


USE_STDPERIPH_DRIVER,STM32F10X_HD

启动文件	区别
startup_stm32f10x_ld.s	ld: low-density 小容量, FLASH 容量在 16-32K 之间
startup_stm32f10x_md.s	md: medium-density 中容量, FLASH 容量在 64-128K 之间
startup_stm32f10x_hd.s	hd: high-density 中容量, FLASH 容量在 256-512K 之间
startup_stm32f10x_xl.s	xl: 超大容量, FLASH 容量在 512-1024K 之间
以上四种都属于基本型, 包括 STM32F101xx、STM32F102xx、STM32F103xx 系列	
startup_stm32f10x_cl.s	cl:connectivity line devices 互联型, 特指 STM32F105xx 和 STM32F107xx 系列
startup_stm32f10x_ld_vl.s	vl:value line devices 超值型系列, 特指 STM32F100xx 系列
startup_stm32f10x_md_vl.s	
startup_stm32f10x_hd_vl.s	

宏	MCU 型号	FLASH大小
STM32F10X_LD	STM32F101xx STM32F102xx STM32F103xx	16 ~ 32 Kbytes
STM32F10X_MD	STM32F101xx STM32F102xx STM32F103xx	64 ~ 128 Kbytes
STM32F10X_HD	STM32F101xx STM32F103xx	256 ~ 512 Kbytes
STM32F10X_CL	STM32F105xx STM32F107xx	忽略



